applied sciences

MDPI

Article

An Efficient Palette Generation Method for Color Image Quantization

Shu-Chien Huang

Department of Computer Science, National Pingtung University, Pingtung City, Pingtung County 90003, Taiwan; schuang@mail.nptu.edu.tw

Abstract: This article describes an efficient method to generate a color palette for color image quantization. The method consists of two stages. In the first stage, the initial palette is generated. Initially, the color palette is an empty set. First, the N colors are generated according to the data distribution of the input image in the RGB (Red, Green, Blue) color space. Then, one color is selected from the N colors and this color is added to the initial palette, and the step is repeated until the color number of the initial palette is equal to K. In the second stage, the quantized image is generated using the fast K-means algorithm. There are many sampling rates used in this study. For each sampled pixel, a fast searching method is employed to efficiently determine the closest color in the palette. Experimental results show that the high-quality quantized images can be generated by the proposed method. When the sampling rate equals 0.125, the computation time of the proposed method is less than 0.3 s for all cases.

Keywords: color image quantization; color palette generation; image compression; fast K-means algorithm

Citation: Huang, S.-C. An Efficient Palette Generation Method for Color Image Quantization. *Appl. Sci.* **2021**, *11*, 1043. https://doi.org/10.3390/app11031043

Received: 17 December 2020
Accepted: 20 January 2021
Published: 24 January 2021

1. Introduction

Nowadays, RGB color images are widely used for storage, transmission and display. In order to reduce the storage space required and the transfer time of the color images, several color image quantization techniques have been proposed [1]. Color image quantization consists of two procedures. The first is to design a color palette which is a set of representative colors, while the second is to map each image pixel to one color in the color palette.

The key to color image quantization is a good color palette. If a good color palette is used, good reconstructed image quality of the compressed color image can be achieved. Several palette design methods have been proposed for color image quantization. Orchard and Bouman [2] proposed the binary splitting approach for color image quantization. Wu [3] employed an iterative process to divide the color space into boxes based on variance minimization. Then, the average value of each resulting box is represented as a color of the palette. Pei and Lo [4] proposed a color quantization technique which is based on the Kohonen neural network to train the color images. Hsieh and Fan [5] calculated the sorted histogram list, and an adaptive clustering algorithm used this list to extract the palette colors for color image quantization. Omran et al. [6] developed a color image quantization algorithm that combines particle swarm optimization with the K-means clustering algorithm. Ghanbarian et al. [7] proposed the ant colony algorithm for color reduction. Hu and Su [8] employed two test conditions to accelerate the K-means algorithm for color image quantization. Hu et al. [9] proposed two schemes to design the color palette. The first scheme employed the splitting-based technique for initial palette generation, and the second employed the k-means algorithm to generate the final color palette. Celebi [10] introduced fast variants of k-means with several initialization schemes for color image quantization. Celebi et al. [11] introduced an effective color quantization

method which is based on divisive hierarchical clustering and the binary splitting strategy. El-Said [12] proposed an optimized Fuzzy C-means algorithm for color image quantization. Ponti [13] proposed a method which makes a contribution regarding the use of image quantization to dimensionality reduction of visual data. Ueda et al. [14] described a color quantization method using statistical features obtained by linear discriminant analysis and principal component analysis. Frackiewicz et al. [15] proposed a fast color image quantization method based on K-means clustering combined with image sampling. Pérez-Delgado [16] employed the shuffled-frog leaping algorithm for color image quantization. Pérez-Delgado [17] solved the color quantization problem by combining the particle swarm optimization algorithm with the Ant-tree.

Although these methods can generate quantized images, some methods generate poorer images and some consume a great deal of time. The aim of this study is to generate high-quality quantized images with low computational cost. Experimental results show that high-quality quantized images can be generated by the proposed method. When the sampling rate equals 0.125, the computation time of the proposed method is less than 0.3 s for all cases.

The rest of this paper is organized as follows. Section 2 presents the related work. Section 3 gives a description of the color image quantization method. In Section 4, the experimental results and discussion are presented. Section 5 gives the conclusions.

2. Related Work

Vector quantization has been successfully used in image compression. There are many techniques which have been proposed to speed up the codebook search in vector quantization [18–25]. Wu and Lin [22] proposed a kick-out condition for the fast codeword searching algorithm. In this article, this fast searching method is called by Wu–Lin method. For a color pixel in the color image, the Wu–Lin method can efficiently determine the closest color in the palette.

For the color pixel $x = (x_1, x_2, x_3)$ in an image and a color $y = (y_1, y_2, y_3)$ in the palette, the square Euclidean distance is defined as follows:

$$\mathrm{SED}(\mathrm{x},\mathrm{y}) = \sum_{\mathrm{j}=1}^{3}\left(\mathrm{x_j} - \mathrm{y_j}\right)^2 = ||\mathrm{x}||^2 + ||\mathrm{y}||^2 - 2\sum_{\mathrm{j}=1}^{3}\mathrm{x_j y_j} \tag{1}$$

The goal is to find a palette color y such that it minimizes Equation (1). Given the color pixel x, $||x||^2$ is a common term for all palette colors. Therefore, the goal is also to find a palette color y such that it minimizes

$$\mathrm{SED1}(x,y) = ||y||^2 - 2\sum_{j=1}^{3} x_j y_j \tag{2}$$

According to the Cauchy–Schwarz inequality, we have

$$||x|| \times ||y|| \geq \sum_{j=1}^{3} x_j y_j \tag{3}$$

Therefore, the following inequality is always true:

$$\mathrm{SED1}(x,y) \geq ||y||^2 - 2||x|| \times ||y|| = ||y||(||y|| - 2||x||) \tag{4}$$

If a palette color y satisfies the test condition

$$||y||(||y|| - 2||x||) \geq SED1\_min \tag{5}$$

where $SED1\_min$ denotes the minimal SED1 between the color pixel x and the closest palette color $y_{\min}$ found so far, then $SED1(x, y) \geq SED1\_min$ can be derived from Equations (4) and (5). Therefore, the palette color y is not the closest color.

The color palette is sorted in ascending order of length. Assume that $c1$ and $c2$ are two colors in the palette. The following two properties are important. The first property is that if $||c1|| < ||x||$ when the palette color $c1$ is not the closest color; the palette color $c2$ is also not the closest color if $||c2|| < ||c1||$. The second property is that if $||c1|| > ||x||$ when the palette color $c1$ is not the closest color; the palette color $c2$ is also not the closest color if $||c2|| > ||c1||$.

If the color pixel x = (125, 65, 130), the value of $||x||$ is about 191.70. Figure 1a shows the sorted palette of K colors. Assume that the color c is the initial searched palette color for the color pixel x. In this study, the color c is the palette color with the closest length value to x, and the binary search technique is employed to find the color c. In this case, the palette color c = (73, 58, 167). It can be computed that the value of $||c||$ is about 191.26. Therefore, Figure 1b shows the searching order of colors in the palette.

(56,10,74)	
(68,23,88)	
⋮	⋮
(73,54,152)	5
(87,70,151)	3
(73,58,167)	1
(133,74,133)	2
(76,69,181)	4
⋮	⋮
(152,201,244)	
(183,198,229)	

(**a**) (**b**)

Figure 1. (**a**) The color palette is sorted in ascending order of length, (**b**) the searching order of colors in the palette if the color pixel x = (125, 65, 130).

3. The Proposed Method

The method consists of two stages: initial palette generation and fast K-means algorithm.

3.1. Initial Palette Generation

For the color pixel $x = (x_1, x_2, x_3)$ in an image, x_1, x_2, and x_3 are integers between 0 and 255. Figure 2 shows the data distribution of the Airplane image in the RGB color space. The color pixel $x = (x_1, x_2, x_3)$ in the Airplane image corresponds to a point (x_1, x_2, x_3) in Figure 2. The 3D RGB space can be divided into non-overlapping cubes. In this study, each cube size is $16 \times 16 \times 16$ such that there are 4096 cubes in total. These cubes are identified as Cube(i), $i = 0, 1, \ldots, 4095$. For the initial palette generation, the proposed method only considers the cubes in which the point number contained in them is greater than or equal to Thr. These considered cubes are identified as mCube(i), $i = 0, 1, \ldots, N-1$. The point number contained in mCube(i) and the center of all the points in mCube(i) are denoted by

initn(i) and initc(i), respectively. For example, the point number contained in mCube(0) is equal to 3. Assume that the locations of these points are (81, 115, 35), (85, 118, 40) and (90, 116, 36). Hence, initc(0) = $\frac{(81, 115, 35) + (85, 118, 40) + (90, 116, 36)}{3}$ = (85, 116, 37).

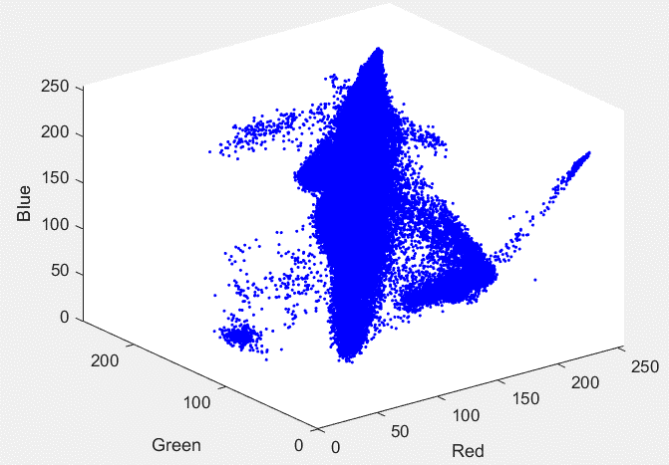


Figure 2. Data distribution of the Airplane image in the RGB (Red, Green, Blue) color space.

Figure 3 shows 8 cubes. The 8 cubes contain 0, 1, 15, 1, 0, 16, 1, and 0 points, respectively. Assume that the value of *Thr* is set to 6. The point numbers contained within Cubes 2 and 5 are greater than or equal to 6. Hence, mCube(0) = Cube(2), mCube(1) = Cube(5), N = 2, initn(0) = 15, and initn(1) = 16. Consequently, initc(0) and initc(1) denote the centers of these points contained in mCube(0) and mCube(1), respectively.

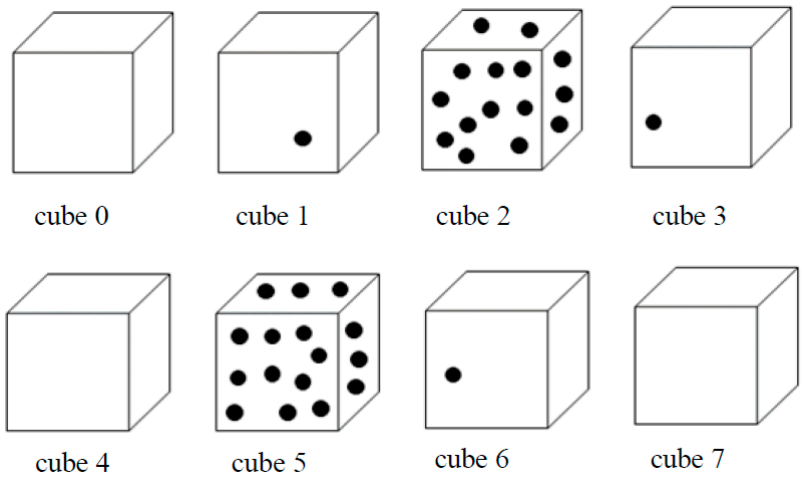


Figure 3. The 8 cubes contain 0, 1, 15, 1, 0, 16, 1, and 0 points, respectively.

After the N colors initc(i), i = 0, 1, . . . , N−1, are determined, the proposed method selects K colors from the N colors to generate the initial palette. The steps of initial palette generation are as follows.

Step 1: The initial palette is an empty set.

Selected(i) = 0, i = 0, 1, . . . , N−1. The value of Selected(i) is equal to 1 which indicates that the color initc(i) has been selected and added to the initial palette.

Cno = 0.

Step 2: If initn(j) == max {initn(i) | i = 0, 1, . . . , N−1}, then the color initc(j) is selected and added to the initial palette.

Selected(j) = 1 and *Cno* = *Cno* +1.

Step 3: For the colors initc(i) with Selected(i) = 0, compute the square Euclidean distance between the color initc(i) and each color in the initial palette. The minimum value among these square Euclidean distances is identified as Dist(i). For the color initc(i), the value DistN(i) is computed as follows

$$\text{DistN}(i) = \text{Dist}(i) \times (\text{initn}(i))^{0.5}. \quad (6)$$

If DistN(j) == max {DistN(i) | if Selected(i) == 0, i = 0, 1, . . . , $N-1$}, then the color initc(j) is selected and added to the initial palette.

Selected(j) = 1 and Cno = Cno + 1.

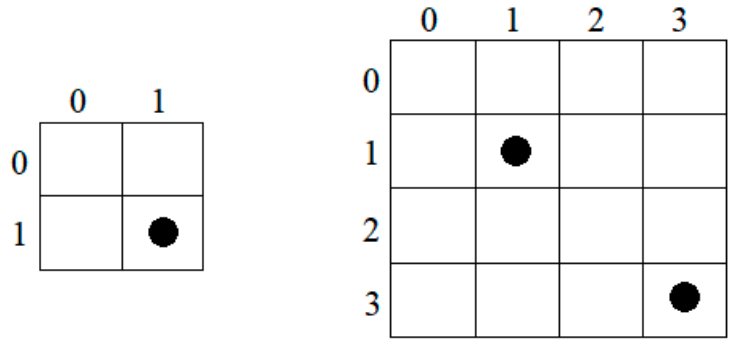
Step 4: If $K > Cno$, then go to Step 3. Otherwise go to Step 5.

Step 5: The initial palette is generated.

3.2. Fast K-Means Algorithm

The K-means algorithm is the common data clustering method. However, the K-means algorithm is time consuming. In order to reduce the computational cost, the fast K-means algorithm is developed.

The data sampling process is employed, and many sampling rates are used in this study. The sampling rate = 1 indicates that every pixel of the color image is sampled. For the 2 × 2 block shown in Figure 4a, only one pixel is sampled from the 2 × 2 block if the sampling rate = 0.25. In this study, a random number, denoted by *rnum1*, is generated. If the value of (*rnum1*% 4) is equal to 3, then the location of the sampled pixel is (3/2, 3% 2) = (1, 1). For the 4×4 block shown in Figure 4b, two pixels are sampled from the 4 × 4 block if the sampling rate = 0.125. In this study, a random number, denoted by *rnum2*, is generated. If the value of (*rnum2*% 8) is equal to 5, then the location of the first sampled pixel is ($x1$, $y1$) = (5/4, 5% 4) = (1, 1) and the location of the second sampled pixel is ($x2$, $y2$) = ($x1$ + 2, ($y1$ + 2)% 4) = (3, 3).



(**a**) (**b**)

Figure 4. (**a**) Only one pixel is sampled, (**b**) two pixels are sampled

According to the data sampling process described above, the sampling rates 0.5, 0.0625 and 0.03125 can also be used. Two pixels are sampled from the 2 × 2 block if the sampling rate = 0.5, only one pixel is sampled from the 4 × 4 block if the sampling rate = 0.0625, and two pixels are sampled from the 8 × 8 block if the sampling rate = 0.03125.

The steps of the fast K-means algorithm can be described as follows.

Input: The initial palette described in Section 3.1.

Step 1: The data sampling process is employed. The sampled color pixels are denoted by SCP_i, i = 0, 1, . . . , SPN—1.

Iter = 0

StopF = 0

Step 2: For the sampled pixel SCP_i, the closest color in the palette is denoted by CCP_i. The color CCP_i is efficiently determined by the Wu–Lin method described in Section 2. If the

index value of its closest palette color is r, then the sampled pixel is classified into the r-th group.
Step 3: Compute the mean values of these K groups. The new color palette is generated by sorting these K values in ascending order of length.
Step 4: The mean square error at iteration *Iter* is defined as follows

$$MSE1(Iter) = \frac{1}{SPN} \sum_{i=0}^{SPN-1} SED(SCP_i, CCP_i). \quad (7)$$

If ((*Iter* >0) and ($MSE1(Iter) \geq MSE1(Iter-1)$)), then set *StopF* to 1.
Step 5: *Iter* = *Iter* + 1.
If ((*Iter* == *Max_cycle*) or (*StopF* == 1)), then go to Step 6. Otherwise, go to Step 2.
Step 6: The color palette is generated. The required iteration number, denoted by *Iter*, is output.

4. Experimental Results and Discussion

The proposed algorithm is implemented in C language. All of the experiments were performed on a PC running under the operating system of Windows 7, with an I7 processor (3.6 GHz) and 32 GBytes of RAM. The set of test images includes Lena, Baboon, Lake, Peppers and Airplane, which have the same size of 512 × 512 pixels. They were downloaded from [26] and were used for conducting the experiments. The parameters *Thr* = 10 and *Max_cycle* = 20 were used in the experiments.

The experimental results are shown in Table 1. The results presented are the solutions with the mean square error, the required iteration number, and computation time. The mean square error (MSE) is defined as follows:

$$MSE = \frac{1}{512 \times 512} \sum_{i=0}^{511} \sum_{j=0}^{511} SED\big(f(i,j), f'(i,j)\big), \quad (8)$$

where f is the original color image, and f' is the quantized image. For the same image and same color palette size, we can observe the following:

(1) The mean square errors obtained when the sampling rate = 1, 0.5, 0.25 and 0.125 are near, and the mean square error obtained when the sampling rate = 0.03125 is higher than that obtained when the sampling rate = 1, 0.5, 0.25, 0.125 and 0.0625 in most cases.
(2) When the sampling rate decreases, the computation time decreases in most cases.

Table 2 shows the computation time (milliseconds) of the proposed method with a sampling rate of 0.125. The total computation time is the summation of T1 and T2. T1 is the computation time for initial palette generation and T2 is the computation time for clustering by the fast K-means algorithm. It is observed that the value of T1 is between 10 and 30 milliseconds, the value of T2 is between 20 and 200 milliseconds, and the total computation is between 30 and 230 milliseconds. For the Lena image, the required iteration numbers are 18 and 13 for K = 128 and 256, respectively. Therefore, the results show that the total computation time when K = 256 is less than the total computation time when K = 128.

Figures 5–9 illustrate the quantized images of Lena, Baboon, Lake, Peppers, and Airplane, respectively. The color numbers of the palette range from 32 to 256. These quantized images are produced by the proposed method with a sampling rate of 0.125. It shows that as the color number of the palette increases, the quality of the quantized image produced by the proposed method always improves. It is noted that the visual quality of the 256 colors quantized image is approximately the same as that of the original image.

Table 1. Results of the proposed method with *Thr* = 10 and *Max_cycle* = 20. (Sr: sampling rate, Iter: the required iteration number, T: computation time (milliseconds)).

		***K* = 32**			***K* = 64**			***K* = 128**			***K* = 256**		
Image	**Sr**	**MSE**	**Iter**	**T**	**MSE**	**Iter**	**T**	**MSE**	**Iter**	**T**	**MSE**	**Iter**	**T**
Lena	1	123.10	13	410	74.00	15	590	47.28	14	710	31.30	20	1260
	0.5	123.39	12	200	73.51	19	390	47.43	17	440	31.45	20	660
	0.25	123.31	14	130	74.12	11	130	47.60	12	170	31.73	20	340
	0.125	123.84	11	60	74.31	16	100	47.86	18	130	32.24	13	120
	0.0625	123.83	19	50	73.62	20	70	48.16	18	80	32.61	14	70
	0.03125	123.96	14	30	74.55	17	40	48.42	19	50	33.55	11	40
Baboon	1	381.87	20	780	240.25	11	550	153.40	14	880	100.48	3	280
	0.5	385.98	20	410	240.51	14	380	153.83	14	460	99.29	16	670
	0.25	385.89	20	210	240.66	12	180	153.99	18	300	99.69	13	290
	0.125	383.92	20	120	241.11	20	150	154.91	16	160	100.22	20	230
	0.0625	389.16	20	70	241.94	16	70	156.29	20	110	101.58	20	130
	0.03125	385.79	20	50	242.84	20	50	157.70	20	70	103.16	20	80
Peppers	1	230.19	20	670	136.73	14	610	84.99	12	660	54.91	14	970
	0.5	230.18	18	320	136.85	15	330	84.94	11	330	55.18	15	550
	0.25	230.58	18	180	136.52	18	210	85.06	14	200	55.47	15	290
	0.125	230.00	14	70	137.14	18	120	85.87	14	120	55.70	14	150
	0.0625	230.56	20	60	137.75	16	60	86.09	11	60	56.53	14	80
	0.03125	232.12	20	40	138.20	19	40	87.12	20	50	58.12	12	50
Lake	1	210.98	20	640	135.05	10	400	86.95	16	800	56.78	20	1220
	0.5	210.62	20	340	134.88	10	220	87.02	14	370	57.33	17	550
	0.25	211.01	20	170	134.89	15	170	87.45	13	190	57.56	20	320
	0.125	211.57	20	100	135.67	11	70	87.99	16	120	58.44	14	130
	0.0625	210.11	17	50	136.19	14	50	88.54	20	80	59.01	18	100
	0.03125	211.40	18	30	138.43	18	40	90.86	20	50	61.14	20	60
Airplane	1	69.43	9	220	41.47	6	170	26.99	8	290	18.55	6	280
	0.5	69.21	14	180	41.49	6	100	26.97	7	140	18.80	8	190
	0.25	69.59	10	80	41.52	6	70	26.90	7	80	18.78	10	120
	0.125	70.21	14	50	41.62	6	30	27.27	7	40	19.22	8	50
	0.0625	70.06	16	40	41.83	6	20	27.59	6	30	18.86	8	40
	0.03125	71.51	10	20	40.68	9	20	27.60	9	20	19.12	9	30

Table 2. The computation time (milliseconds) of the proposed method with a sampling rate of 0.125.

Image	**K**	**T1**	**T2**	**Total Computation Time**
Lena	32	10	50	60
	64	10	90	100
	128	10	120	130
	256	10	110	120
Baboon	32	20	100	120
	64	30	120	150
	128	30	130	160
	256	30	200	230
Peppers	32	10	60	70
	64	20	100	120
	128	20	100	120
	256	20	130	150
Lake	32	10	90	100
	64	10	60	70
	128	20	100	120
	256	20	110	130
Airplane	32	10	40	50
	64	10	20	30
	128	10	30	40
	256	10	40	50

Figure 5. (**a**) The Lena image, (**b**) 32 colors with MSE = 123.84, (**c**) 64 colors with MSE = 74.31, (**d**) 128 colors with MSE = 47.86, (**e**) 256 colors with MSE = 32.24.

(**a**)

(**b**)

(**c**)

(**d**)

(**e**)

Figure 6. (**a**) The Baboon image, (**b**) 32 colors with MSE = 383.92, (**c**) 64 colors with MSE = 241.11, (**d**) 128 colors with MSE = 154.91, (**e**) 256 colors with MSE = 100.22.

(**a**)

(**b**)

(**c**)

(**d**)

(**e**)

Figure 7. (**a**) The Pepper image, (**b**) 32 colors with MSE = 230.00, (**c**) 64 colors with MSE = 137.14, (**d**) 128 colors with MSE = 85.87, (**e**) 256 colors with MSE = 55.70.

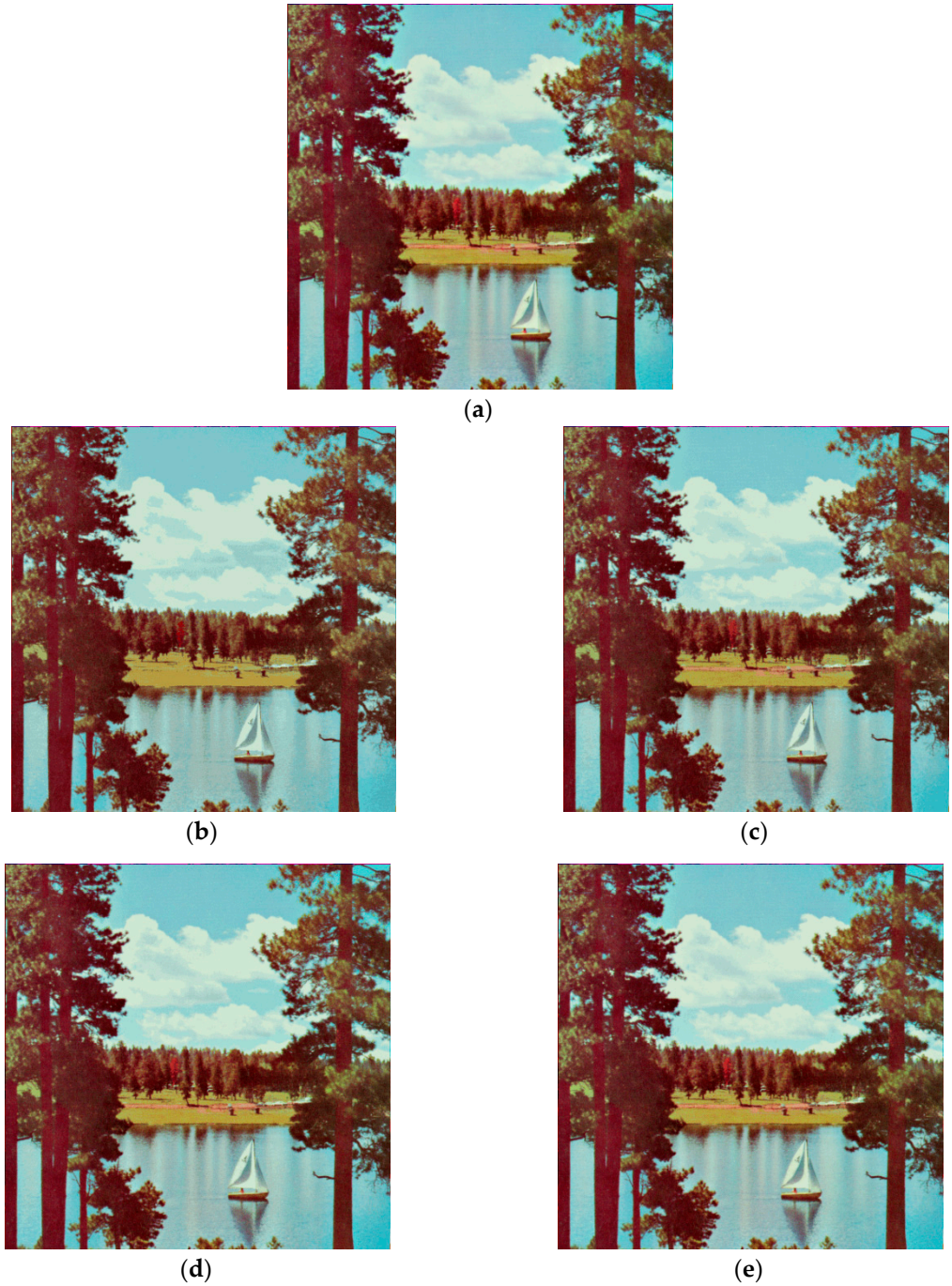
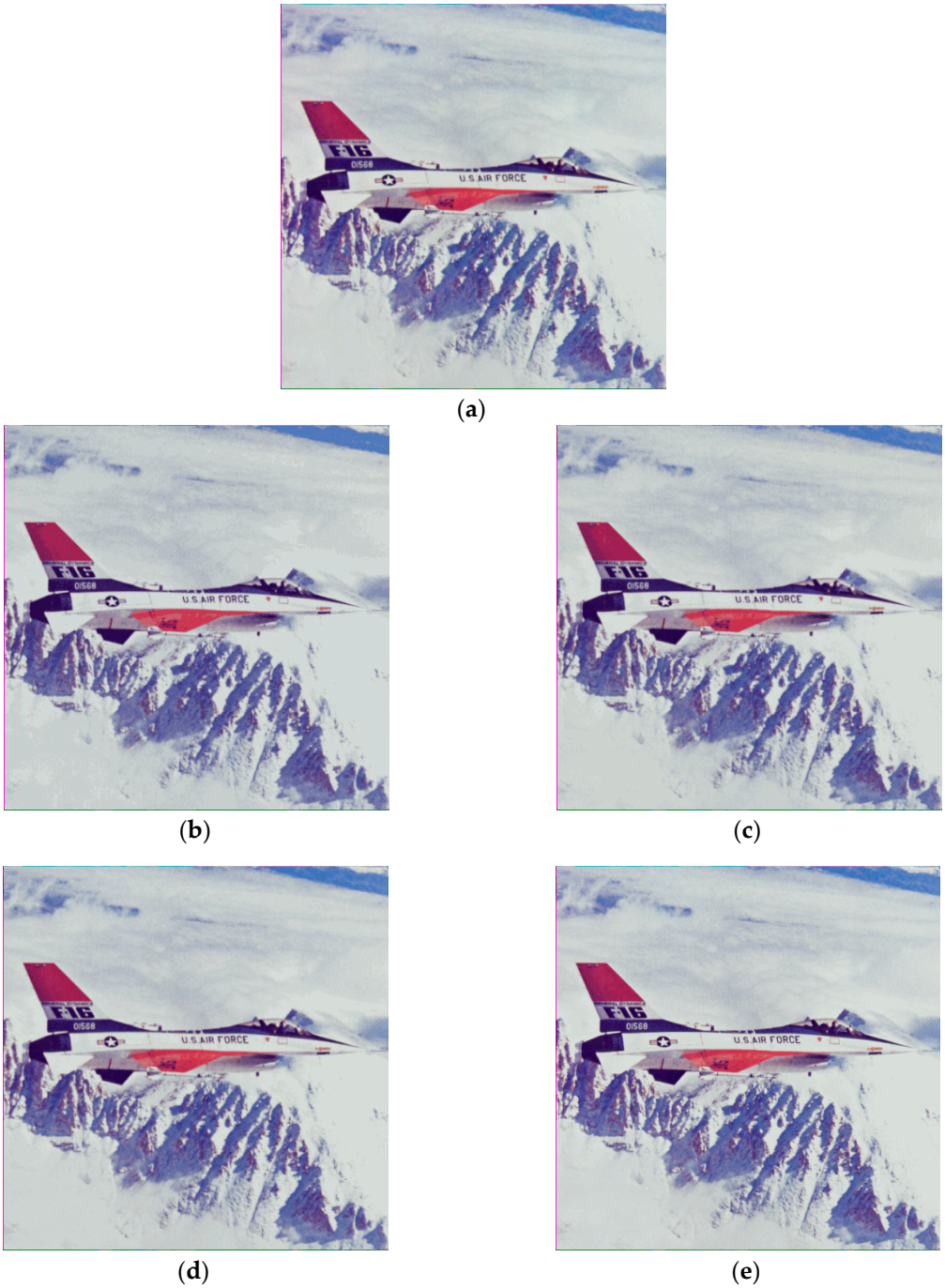


Figure 8. (**a**) The Lake image, (**b**) 32 colors with MSE = 211.57, (**c**) 64 colors with MSE = 135.67, (**d**) 128 colors with MSE = 87.99, (**e**) 256 colors with MSE = 58.44.



(**a**) (**b**) (**c**) (**d**) (**e**)

Figure 9. (**a**) The Airplane image, (**b**) 32 colors with MSE = 70.21, (**c**) 64 colors with MSE = 41.62, (**d**) 128 colors with MSE = 27.27, (**e**) 256 colors with MSE = 19.22.

In addition, two color images, Splash and Tree, were used for testing. The Splash image consists of 512 × 512 pixels, and the Tree image consists of 256 × 256 pixels. Figures 10 and 11 illustrate the quantized images of Splash and Tree, respectively. These quantized images are produced by the proposed method with a sampling rate of 0.125. The results show that the proposed method can generate high-quality quantized images. It is observed that the visual quality of the 256 colors quantized image is approximately the same as that of the original image.

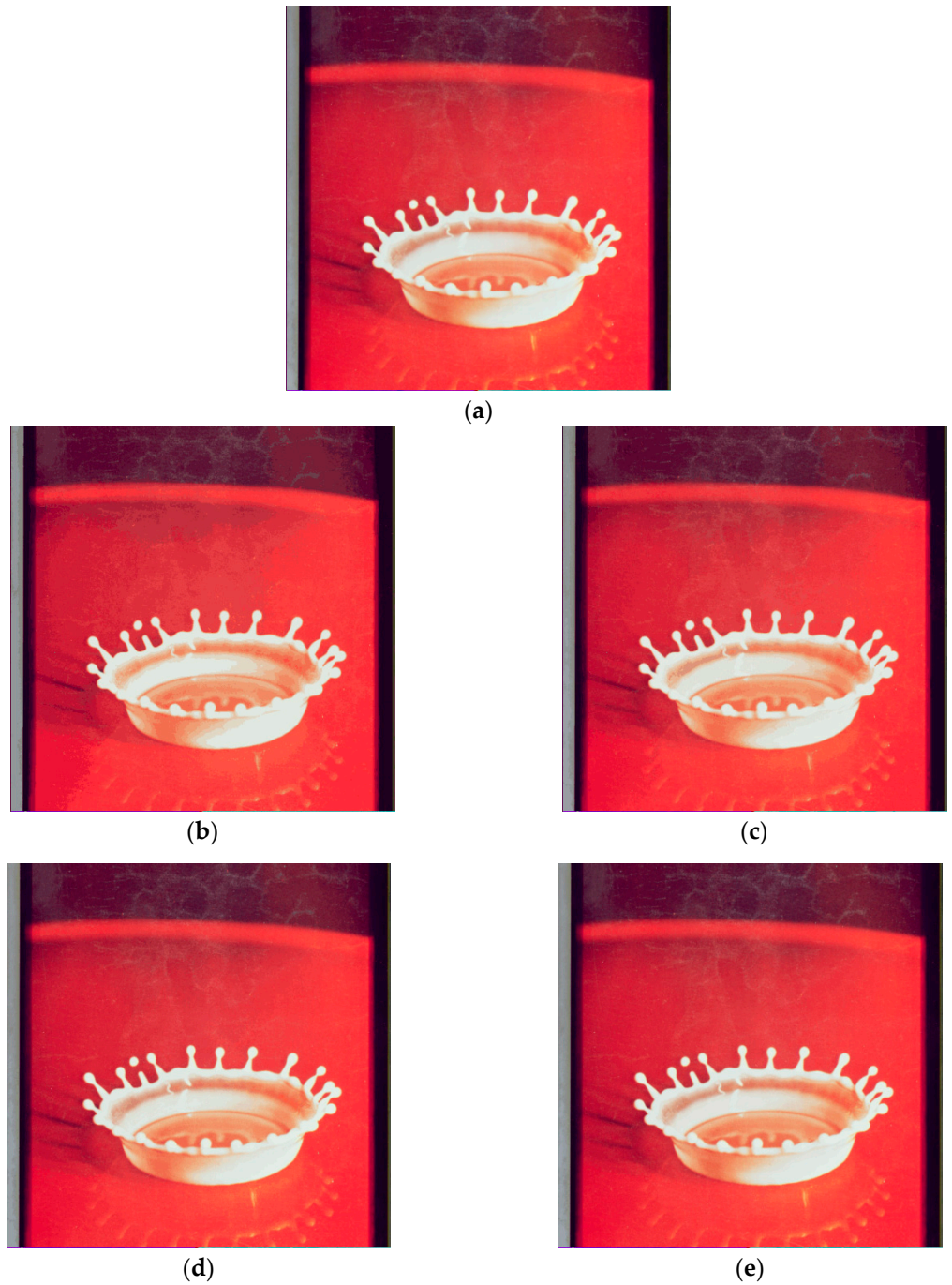


Figure 10. (**a**) The Splash image, (**b**) 32 colors with MSE = 96.72, (**c**) 64 colors with MSE = 52.64, (**d**) 128 colors with MSE = 30.65, (**e**) 256 colors with MSE = 19.31.

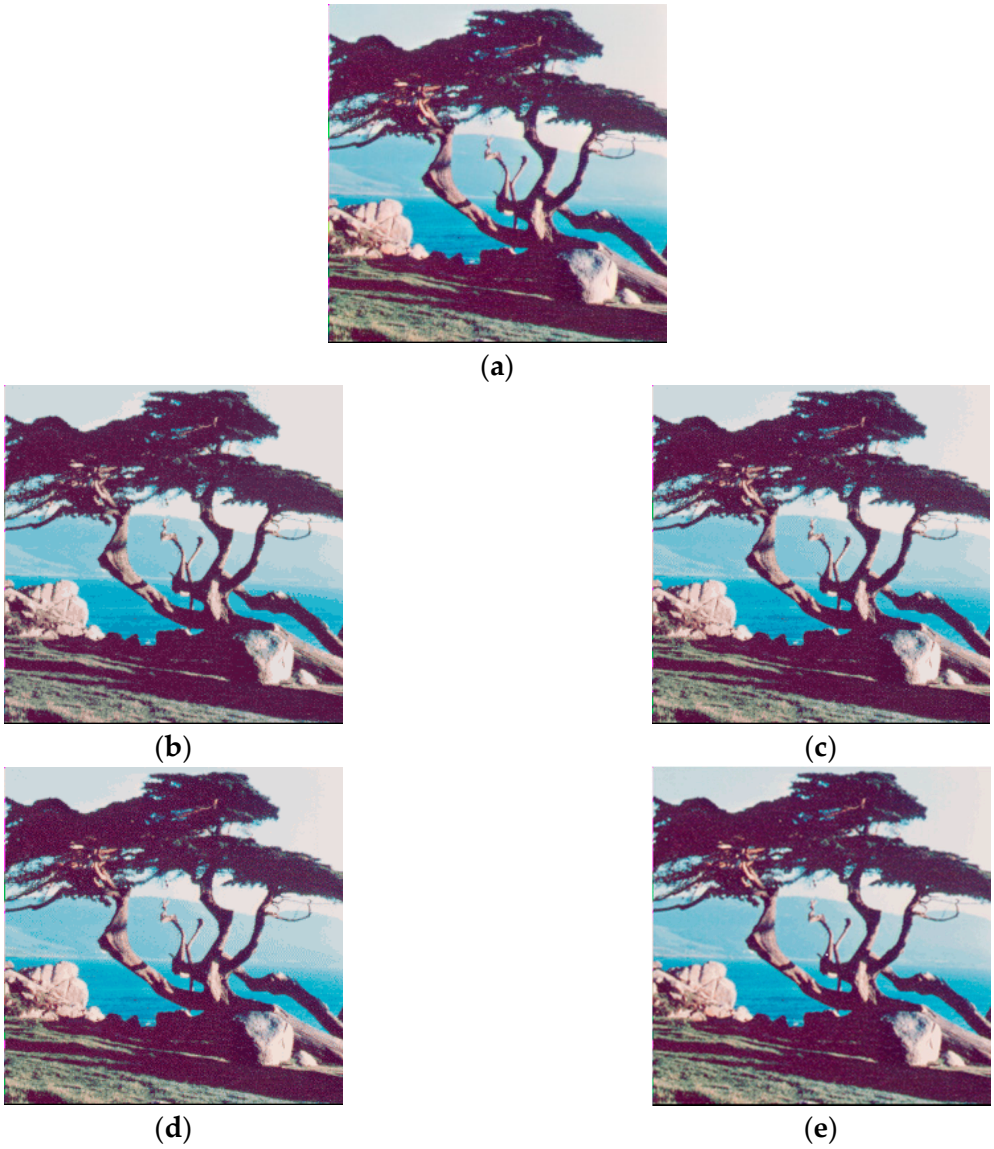


Figure 11. (**a**) The Tree image, (**b**) 32 colors with MSE = 34.50, (**c**) 64 colors with MSE = 20.92, (**d**) 128 colors with MSE = 13.79, (**e**) 256 colors with MSE = 9.44.

The effect of the parameter *Thr* on the solution is discussed in this section. The comparison is applied to the Baboon and Peppers images. The MSE obtained will be shown in a figure for comparison.

The proposed method only considers the cubes for which the point number contained in them is greater than or equal to *Thr* in order to generate the initial palette. Figure 12 compares the results of *Thr* = {3, 10, 30, 60} for the Baboon image. This figure shows that the mean square errors obtained when *Thr* = 3, 10 and 30 are near, and the average mean square error obtained when *Thr* = 60 is slightly higher than that obtained when *Thr* = 3, 10 and 30. Figure 13 compares the results of *Thr* = {3, 10, 30, 60} for the Peppers image. This figure shows that the average mean square error obtained when *Thr* = 10 is slightly lower than that obtained when *Thr* = 3 and 30, and the average mean square error obtained when *Thr* = 60 is higher than that obtained when *Thr* = 3, 10 and 30. Therefore, the value selected for the parameter *Thr* of the proposed algorithm was 10.

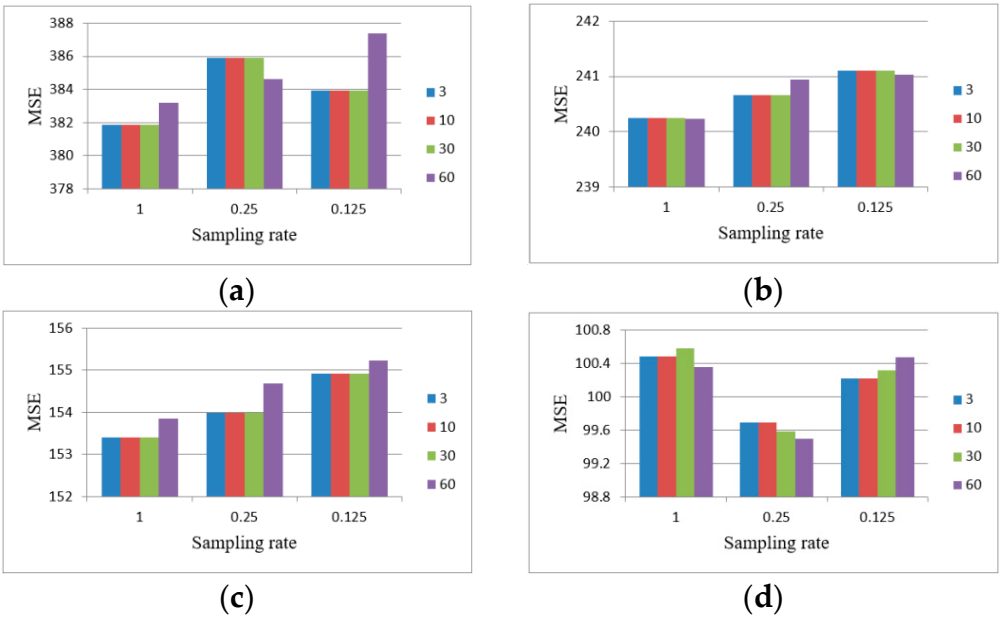


Figure 12. Compare the results of *Thr* = {3, 10, 30, 60} for the Baboon image, (**a**) MSE—32 colors, (**b**) MSE—64 colors, (**c**) MSE—128 colors, (**d**) MSE—256 colors.

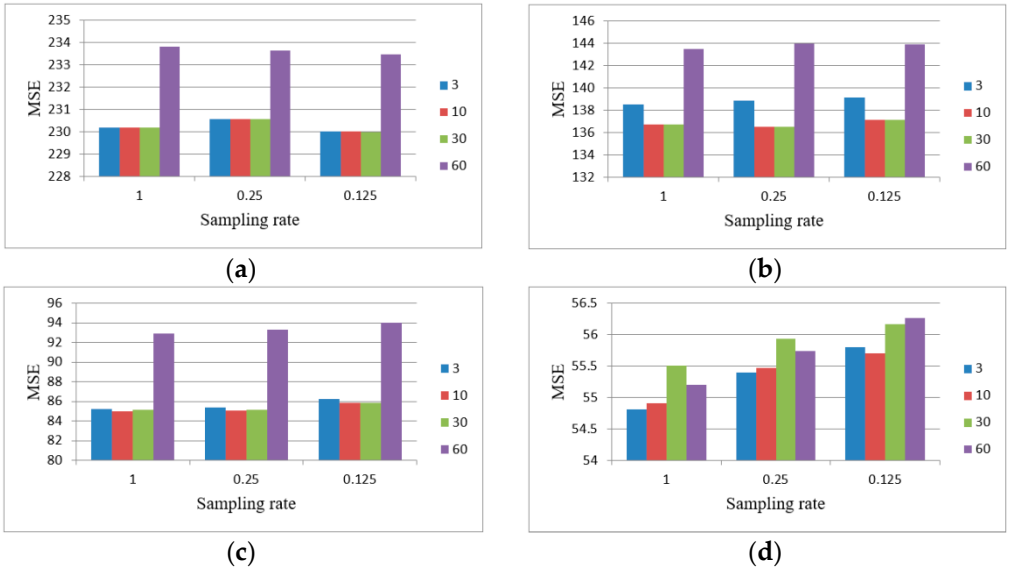


Figure 13. Compare the results of *Thr* = {3, 10, 30, 60} for the Peppers image, (**a**) MSE—32 colors, (**b**) MSE—64 colors, (**c**) MSE—128 colors, (**d**) MSE—256 colors.

In 2020, a color image quantization algorithm, identified as PSO+ATCQ-3, that combines particle swarm optimization and artificial ants was proposed by Pérez-Delgado [17]. The quality of the quantized images produced by PSO+ATCQ-3 is better than that produced by some well-known color image quantization methods. The PSO+ATCQ-3 algorithm was implemented in C language and executed on a PC running under the Linux operating system with an I7 processor (2 GHz) and 16 GBytes of RAM [17]. The experimental results obtained by PSO+ATCQ-3 are shown in Table 3.

First, the performance of the proposed method with sampling rate = 0.125 is compared to the iteration 1 of PSO+ATCQ-3. The MSE obtained by PSO+ATCQ-3 is higher than that obtained by the proposed method for all cases. The computation time of PSO+ATCQ-3 is between 468 and 2074 milliseconds. The computation time of the proposed method is

between 30 and 230 milliseconds. It reveals that PSO+ATCQ-3 consumes more computation time than the proposed method.

Second, the performance of the proposed method with sampling rate = 0.25 is compared to the iteration 5 of PSO+ATCQ-3. For the Lena, Baboon, Peppers and Lake images, the MSE obtained by PSO+ATCQ-3 is slightly lower than that obtained by the proposed method. For the Airplane image, the MSE obtained by PSO+ATCQ-3 is higher than that obtained by the proposed method. The computation time of PSO+ATCQ-3 is between 2358 and 10,412 milliseconds. The computation time of the proposed method is between 70 and 340 milliseconds. It also reveals that PSO+ATCQ-3 consumes more computation time than the proposed method.

Table 3. Results of the method proposed by Pérez-Delgado [17]. (Iter: iteration of the algorithm, MSE_a: average MSE, dev: standard deviation of MSE, T: average computation time (milliseconds)).

		K = 32			K = 64			K = 128			K = 256		
Image	**Iter**	**MSE_a**	**dev**	**T**	**MSE_a**	**dev**	**T**	**MSE_a**	**dev**	**T**	**MSE_a**	**dev**	**T**
Lena	1	126.44	2.09	468	77.30	1.27	723	50.00	0.46	1172	32.77	0.30	2074
	5	119.26	0.61	2373	73.05	0.60	3616	46.96	0.32	5827	30.84	0.21	10412
	10	118.56	0.41	4764	72.53	0.38	7219	46.66	0.27	11787	30.61	0.20	20783
Baboon	1	390.16	3.87	478	247.09	2.29	716	159.21	1.24	1177	103.40	0.64	2042
	5	376.97	1.19	2401	238.57	0.87	3593	153.75	0.65	5869	98.77	0.39	10209
	10	375.43	0.58	4803	237.79	0.70	7224	153.24	0.53	11712	98.41	0.34	20452
Peppers	1	245.47	4.81	478	148.21	3.19	739	91.79	2.65	1167	59.87	0.88	2049
	5	230.25	1.19	2382	136.12	1.78	3678	84.76	0.81	5894	55.42	0.42	10244
	10	229.31	1.37	4727	134.59	1.51	7325	84.31	0.47	11746	54.66	0.34	20484
Lake	1	216.97	4.06	474	143.66	3.40	712	95.53	1.77	1156	63.32	1.27	2043
	5	204.89	0.76	2358	132.67	0.88	3595	87.01	0.52	5890	56.94	0.33	10223
	10	203.73	0.79	4718	131.69	0.49	7183	85.75	0.43	11735	55.53	0.36	20447
Airplane	1	130.06	17.75	469	67.12	10.42	701	38.70	3.40	1182	24.98	1.06	2055
	5	91.88	14.08	2366	53.00	3.74	3501	33.06	2.22	5874	21.68	0.83	10242
	10	71.61	4.97	4707	46.94	2.31	7004	31.49	1.49	11761	21.08	0.72	20588

5. Conclusions

In this article, an efficient method is presented to generate the color palette. The proposed method consists of two stages. The first stage is to generate the initial palette. First, the N colors are generated according to the data distribution of the input image in the RGB color space. Then, one color is selected from the N colors and this color is added to the initial palette, and the step is repeated until the color number of the initial palette is equal to K. The second stage is to generate the quantized images using the fast K-means algorithm. The data sampling process and the Wu–Lin method are employed to achieve reduction in computation time.

Many sampling rates are used in this study. Experimental results show that the visual quality of the 256 colors quantized image is approximately the same as that of the original image. When the sampling rate = 0.125, the computation time of the proposed method is less than 0.3 s. In other words, the proposed method can generate high-quality quantized images while consuming little computational cost. It is thus suitable for real-time multimedia applications.

Funding: This research received no external funding.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Acknowledgments: This work was partially supported by the Ministry of Science and Technology, Taiwan, under Grant MOST 104-2221-E-153-012.

Conflicts of Interest: The author declares that there is no conflict of interest.

References

1. Ozturk, C.; Hancer, E.; Karaboga, D. Color image quantization: A short review and an application with artificial bee colony algorithm. *Informatica* **2014**, *25*, 485–503. [CrossRef]
2. Orchard, M.T.; Bouman, C.A. Color quantization of images. *IEEE Trans. Signal Process.* **1991**, *39*, 2677–2690. [CrossRef]
3. Wu, X. Efficient Statistical Computations for Optimal Color Quantization. In *Graphics Gems II*; Arvo, J., Ed.; Academic Press: New York, NY, USA, 1991; pp. 126–133.
4. Pei, S.C.; Lo, Y.S. Color image compression and limited display using self-organization Kohonen map. *IEEE Trans. Circuits Syst. Video Technol.* **1998**, *8*, 191–205.
5. Hsieh, I.S.; Fan, K.C. An adaptive clustering algorithm for color quantization. *Pattern Recognit. Lett.* **2000**, *21*, 337–346. [CrossRef]
6. Omran, M.G.; Engelbrecht, A.P.; Salman, A. A color image quantization algorithm based on particle swarm optimization. *Informatica* **2005**, *29*, 261–269.
7. Ghanbarian, A.T.; Kabir, E.; Charkari, N.M. Color reduction based on ant colony. *Pattern Recognit. Lett.* **2007**, *28*, 1383–1390. [CrossRef]
8. Hu, Y.C.; Su, B.H. Accelerated pixel mapping scheme for colour image quantization. *Imaging Sci. J.* **2008**, *56*, 68–78. [CrossRef]
9. Hu, Y.C.; Lee, M.G.; Tsai, P. Colour palette generation schemes for colour image quantization. *Imaging Sci. J.* **2009**, *57*, 46–57. [CrossRef]
10. Celebi, M.E. Improving the performance of k-means for color quantization. *Image Vis. Comput.* **2011**, *29*, 260–271. [CrossRef]
11. Celebi, M.E.; Wen, Q.; Hwang, S. An effective real-time color quantization method based on divisive hierarchical clustering. *J. Real-Time Image Process.* **2015**, *10*, 329–344. [CrossRef]
12. El-Said, S.A. Image quantization using improved artificial fish swarm algorithm. *Soft Comput.* **2015**, *19*, 2667–2679. [CrossRef]
13. Ponti, M.; Nazaré, T.S.; Thumé, G.S. Image quantization as a dimensionality reduction procedure in color and texture feature extraction. *Neurocomputing* **2016**, *173*, 385–396. [CrossRef]
14. Ueda, Y.; Koga, T.; Suetake, N.; Uchino, E. Color quantization method based on principal component analysis and linear discriminant analysis for palette-based image generation. *Opt. Rev.* **2017**, *24*, 741–756. [CrossRef]
15. Frackiewicz, M.; Mandrella, A.; Palus, H. Fast color quantization by K-means clustering combined with image sampling. *Symmetry* **2019**, *11*, 963. [CrossRef]
16. Pérez-Delgado, M.L. Color image quantization using the shuffled-frog leaping algorithm. *Eng. Appl. Artif. Intell.* **2019**, *79*, 142–158. [CrossRef]
17. Pérez-Delgado, M.L. Color quantization with particle swarm optimization and artificial ants. *Soft Comput.* **2020**, *24*, 4545–4573. [CrossRef]
18. Huang, S.H.; Chen, S.H. Fast encoding algorithm for VQ-based image coding. *Electron. Lett.* **1990**, *26*, 1618–1619. [CrossRef]
19. Ra, S.W.; Kim, J.K. A fast mean-distance-ordered partial codebook search algorithm for image vector quantization. *IEEE Trans. Circuits Syst. II Analog Digit. Signal Process.* **1993**, *40*, 576–579. [CrossRef]
20. Torres, L.; Huguet, J. An improvement on codebook search for vector quantization. *IEEE Trans. Commun.* **1994**, *42*, 208–210. [CrossRef]
21. Lin, Y.C.; Tai, S.C. A fast Linde-Buzo-Gray algorithm in image vector quantization. *IEEE Trans. Circuits Syst. II Analog Digit. Signal Process.* **1998**, *45*, 432–435.
22. Wu, K.S.; Lin, J.C. Fast VQ encoding by an efficient kick-out condition. *IEEE Trans. Circuits Syst. Video Technol.* **2000**, *10*, 59–62.
23. Hu, Y.C.; Chang, C.C. An effective codebook search algorithm for vector quantization. *Imaging Sci. J.* **2003**, *51*, 221–234. [CrossRef]
24. Pan, J.S.; Lu, Z.M.; Sun, S.H. An efficient encoding algorithm for vector quantization based on subvector technique. *IEEE Trans. Image Process.* **2003**, *12*, 265–270. [PubMed]
25. Chang, C.C.; Hsieh, Y.P. A fast VQ codebook search with initialization and search order. *Inf. Sci.* **2012**, *183*, 132–139. [CrossRef]
26. The USC-SIPI Image Database. Available online: http://sipi.usc.edu/database/database.php/ (accessed on 15 June 2018).